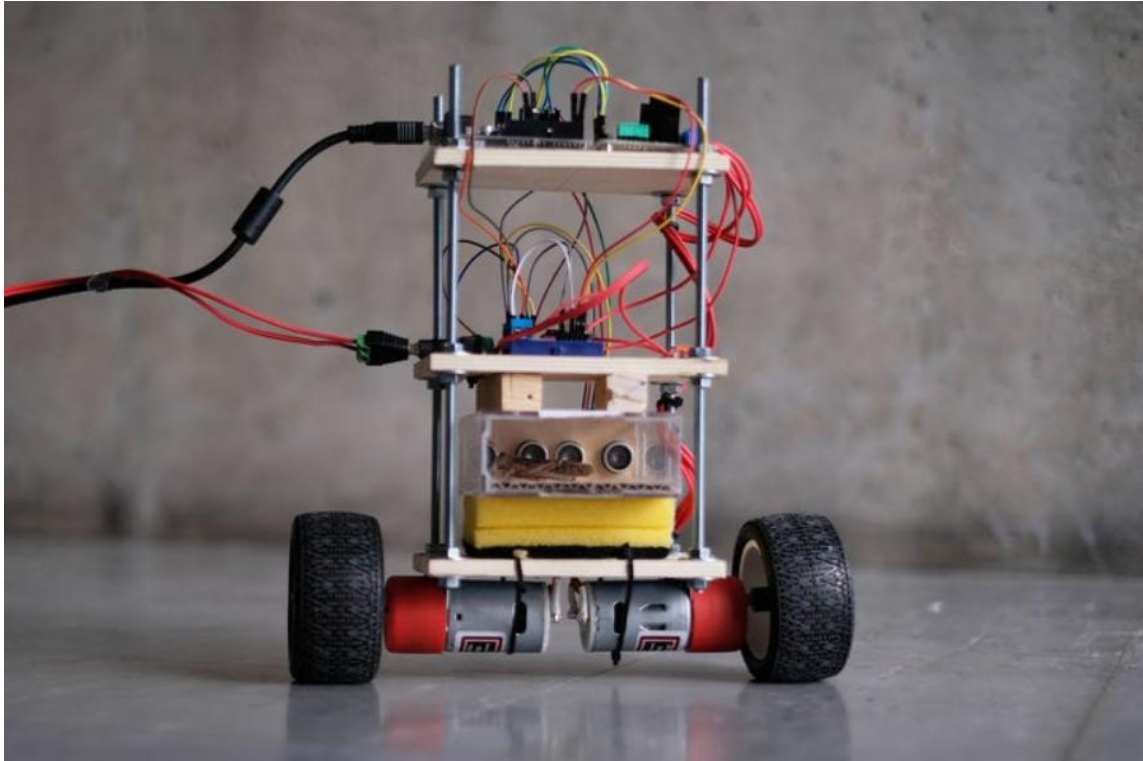


Applied Digital Control - NANENG 212

Self-Balancing Robot - Project



Zewail City of Science & Technology – Spring 2026

Prof. Ahmed Abd-Rabou

Eng. Mohamed Abdelmoneim

Eng. Abdelrahman Yousef

Project by:

Alzahraa Refaat 202401536

Amira Ibrahim 202401172

Haneen Gamil 202400845

Hassan Emad 202401299

MATLAB Data Preprocessing and Object Construction

Command Window

```
>> % 1. Load the data again
data = readmatrix('C:\Users\MacbookPro\Downloads\test 1\test4.txt');

% 2. Remove any row that has a NaN in it
data = rmmissing(data);

% 3. Now split the clean data
u = data(:, 1); % Input
y = data(:, 2); % Output (Angle)

% 4. Create your identification object
z = iddata(y, u, 0.01);
```

Transfer Function of our system

```
G_cl_s =

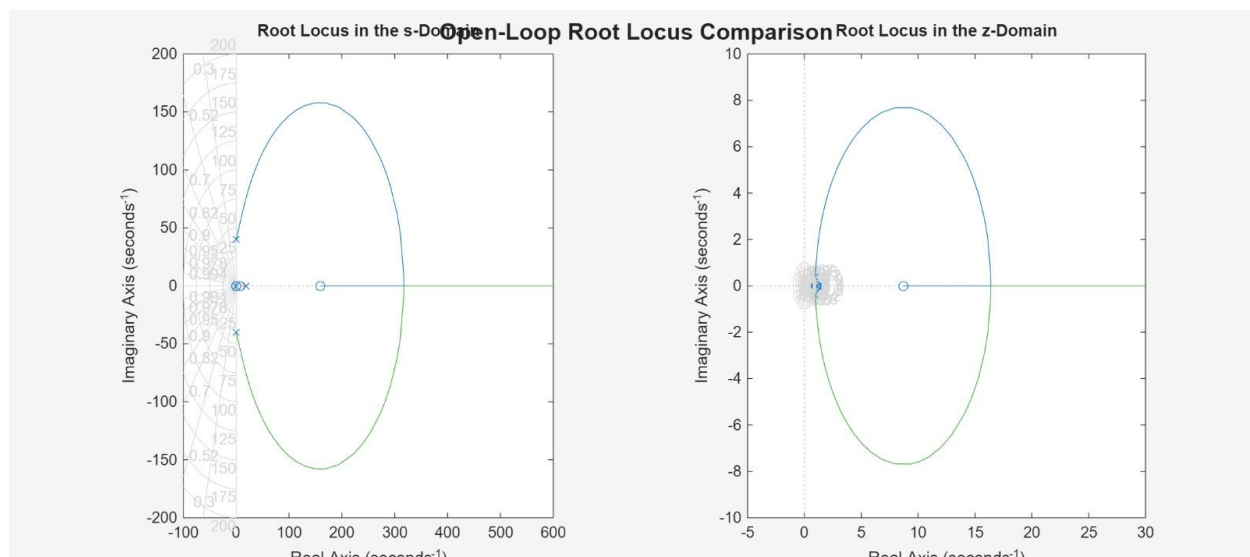
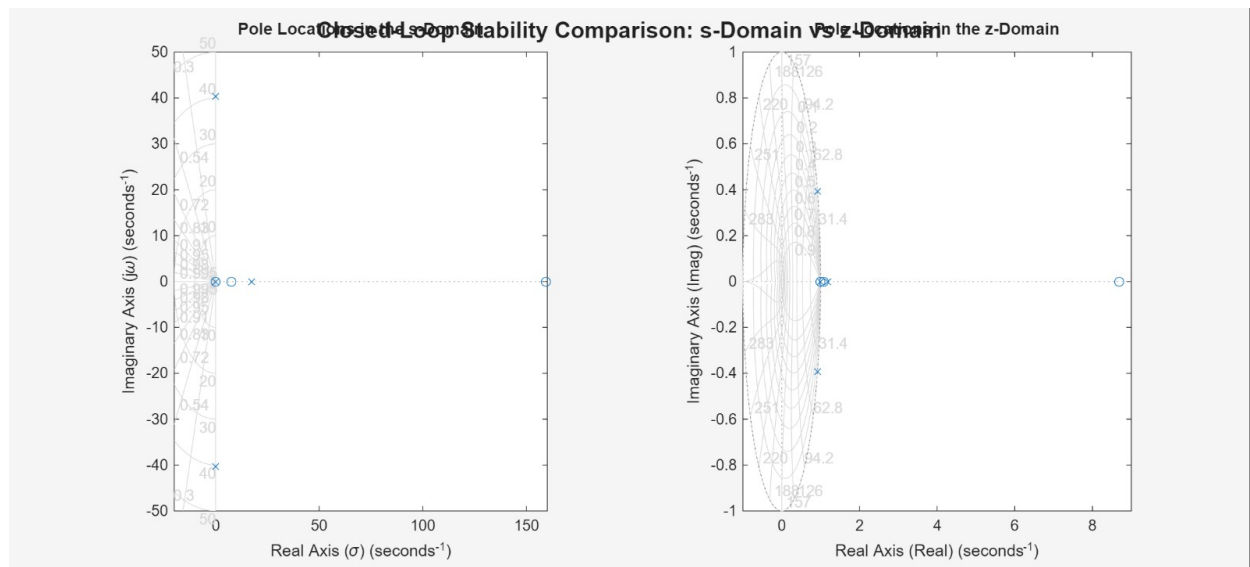
      -0.2549 s^3 - 73.73 s^2 - 2859 s - 10.08
      -----
      s^4 + 26.99 s^3 + 1187 s^2 + 2.794e04 s - 2.329e04
```

With K

```
G_cl_s2 =

      -0.2549 s^3 - 73.73 s^2 - 2859 s - 10.08
      -----
      s^4 + 27.07 s^3 + 1187 s^2 + 2.791e04 s - 2.319e04
```

Without K



Continuous-time identified state-space model:

$$\frac{dx}{dt} = A x(t) + B u(t) + K e(t)$$

$$y(t) = C x(t) + D u(t) + e(t)$$

A =

	x1	x2	x3	x4
x1	-0.02403	21.54	0.3202	-4.391
x2	-22.62	-0.7709	-26.58	18.26
x3	0.3578	25.14	-2.221	24.57
x4	-1.37	-1.421	2.538	-24.05

B =

	u1
x1	-0.0006114
x2	0.01898
x3	-0.00776
x4	0.005432

C =

	x1	x2	x3	x4
y1	-167.2	-17.36	4.039	0.6868

K =

	y1
x1	-0.6298
x2	-2.141
x3	5.1
x4	-0.06791

ans =

-1.2738 +33.8206i
-1.2738 -33.8206i
-25.2436 + 0.0000i
0.8055 + 0.0000i

Open-Loop Poles:

ans =

-1.3081 +33.7910i
-1.3081 -33.7910i
-25.2527 + 0.0000i
0.8029 + 0.0000i

Arduino Code

```
1  #include "LMotorController.h"
2  #include "I2Cdev.h"
3  #include "MPU6050_6Axis_MotionApps20.h"
4  #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
5  |   #include "Wire.h"
6  #endif
7
8  #define MIN_ABS_SPEED 20
9  MPU6050 mpu;
10
11 // MPU control/status vars
12 bool dmpReady = false;
13 uint8_t mpuIntStatus;
14 uint8_t devStatus;
15 uint16_t packetSize;
16 uint16_t fifoCount;
17 uint8_t fifoBuffer[64];
18
19 // orientation/motion vars
20 Quaternion q;
21 VectorFloat gravity;
22 float ypr[3];
23
24 // --- MANUAL DISCRETE PID VARIABLES ---
25 double setpoint = 162.0;
26 double input = 0;
27
28 // MATLAB Tuned Gains (Using the safely scaled Kd)
29 double Kp = 22;
30 double Ki = 155;
31 double Kd = 1.5; // Remember: Scaled down from 7050 for testing!
32 double Ts = 0.01; // Sampling time (100Hz from MPU6050)
33
34 // Discrete difference equation constants
35 double alpha = Ki * Ts;
36 double beta = Kd / Ts;
37
38 // Memory variables for discrete time steps (k-1)
```

```

39 double ek = 0; // Current Error: e(k)
40 double ek_1 = 0; // Previous Error: e(k-1)
41 double pk_1 = 0; // Previous Integral: p(k-1)
42
43 // Output components
44 double wk, qk, pk, uk;
45
46 // MOTOR CONTROLLER
47 double motorSpeedFactorLeft = 0.6;
48 double motorSpeedFactorRight = 0.6;
49 int ENA = 5;
50 int IN1 = 6;
51 int IN2 = 7;
52 int IN3 = 9;
53 int IN4 = 8;
54 int ENB = 10;
55 LMotorController motorController(ENA, IN2, IN1, ENB, IN3, IN4, motorSpeedFactorLeft, motorSpeedFactorRight);
56
57 volatile bool mpuInterrupt = false;
58 void dmpDataReady() {
59     mpuInterrupt = true;
60 }
61
62 void setup()
63 {
64     #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
65         Wire.begin();
66         TWBR = 24;
67     #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
68         Fastwire::setup(400, true);
69     #endif
70
71     Serial.begin(115200);
72     while (!Serial);
73
74     Serial.println(F("Initializing I2C devices..."));
75     mpu.initialize();
76     Serial.println(mpu.testConnection() ? F("MPU6050 connection successful") : F("MPU6050 connection failed"));

```

```

77
78     devStatus = mpu.dmpInitialize();
79
80     // Your gyro offsets
81     mpu.setXGyroOffset(220);
82     mpu.setYGyroOffset(76);
83     mpu.setZGyroOffset(-85);
84     mpu.setZAccelOffset(1788);
85
86     if (devStatus == 0)
87     {
88         Serial.println(F("Enabling DMP..."));
89         mpu.setDMPEnabled(true);
90
91         Serial.println(F("Enabling interrupt detection..."));
92         attachInterrupt(0, dmpDataReady, RISING);
93         mpuIntStatus = mpu.getIntStatus();
94
95         Serial.println(F("DMP ready! Waiting for first interrupt..."));
96         dmpReady = true;
97         packetSize = mpu.dmpGetFIFOPageSize();
98     }
99     else
100     {
101         Serial.print(F("DMP Initialization failed (code "));
102         Serial.print(devStatus);
103         Serial.println(F(")"));
104     }
105 }
106
107 void loop()
108 {
109     if (!dmpReady) return;
110
111     // Wait for the MPU interrupt
112     while (!mpuInterrupt && fifoCount < packetSize) {
113         // Do nothing while waiting for the 100Hz clock pulse

```

```

114     }
115
116     mpuInterrupt = false;
117     mpuIntStatus = mpu.getIntStatus();
118     fifoCount = mpu.getFIFOCount();
119
120     if ((mpuIntStatus & 0x10) || fifoCount >= 1024)
121     {
122         mpu.resetFIFO();
123         Serial.println(F("FIFO overflow!"));
124     }
125     else if (mpuIntStatus & 0x02)
126     {
127         while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();
128
129         mpu.getFIFOBytes(fifoBuffer, packetSize);
130         fifoCount -= packetSize;
131
132         mpu.dmpGetQuaternion(&q, fifoBuffer);
133         mpu.dmpGetGravity(&gravity, &q);
134         mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
135
136         // 1. READ SENSOR
137         input = ypr[1] * 180 / M_PI + 180;
138
139         // --- 2. MANUAL DISCRETE PID COMPUTATION ---
140
141         // Calculate Error
142         ek = setpoint - input;
143
144         // Proportional Term
145         wk = Kp * ek;
146
147         // Derivative Term
148         qk = beta * (ek - ek_1);
149
150         // Integral Term
151         pk = (alpha * ek) + pk_1;
152
153         // Anti-windup logic (limit the integral term so it doesn't build up forever)
154         if (pk > 255) pk = 255;
155         if (pk < -255) pk = -255;
156
157         // Total Controller Output
158         uk = wk + qk + pk;
159
160         // Constrain total output to PWM limits
161         if (uk > 255) uk = 255;
162         if (uk < -255) uk = -255;
163
164         // Save current values for the next discrete time step (k-1)
165         ek_1 = ek;
166         pk_1 = pk;
167
168         // --- 3. COMMAND MOTORS ---
169         motorController.move(uk, MIN_ABS_SPEED);
170
171         // --- 4. DATA LOGGING (Time, Angle) ---
172         unsigned long timeStamp = millis();
173         Serial.print(timeStamp);
174         Serial.print(",");
175         Serial.println(input);
176     }
177 }

```

MATLAB Analysis

MATLAB Assumes a "Frictionless Vacuum"

When you gave MATLAB those A, B, C, and D matrices, it built a mathematically pure universe. It assumes the motors respond instantly to the tiniest fraction of a volt, the gears have zero friction, and the wheels never slip.

Because it thinks the robot is so perfectly responsive, it calculates a very gentle controller ($K_p = 6.57$). But in the real world, your robot has physical resistance, meaning that gentle push is not enough to catch the heavy robot as it falls. You need the muscle of your $K_p = 22$.

```
Command Window
>> % 1. Define State-Space Matrices and your K matrix
A = [-0.1682, -27.51, 0.8027, -4.575; 27.19, -0.302, -29.54, -14.54; -1.4, 28.71, -0.5863, -12.33; -0.501, 2.2, 1.882, 20.05];
B = [0.002785; 0.006545; -0.007771; 0.004579];
C = [-280, 36.72, -11.9, 1.009];
D = 0;
K = [16.5916, 17.5143, -1.8851, 290.4901];

% 2. Build the continuous closed-loop state-space system (A - B*K)
sys_cl_s = ss(A - B*K, B, C, D);
sys_cl_s2 = ss(A, B, C, D);

% 3. Convert directly to the s-domain Transfer Function
G_cl_s = tf(sys_cl_s)
G_cl_s2=tf(sys_cl_s2)
pole(G_cl_s)
pole(G_cl_s2)|
```

Command Window

```
>> pole(G_cl_s)
pole(G_cl_s2)

ans with K =

    0.1118 +40.3101i
    0.1118 -40.3101i
   17.3489 + 0.0000i
   -0.0846 + 0.0000i

ans without K =

    0.1291 +40.3237i
    0.1291 -40.3237i
   18.8200 + 0.0000i
   -0.0846 + 0.0000i
```

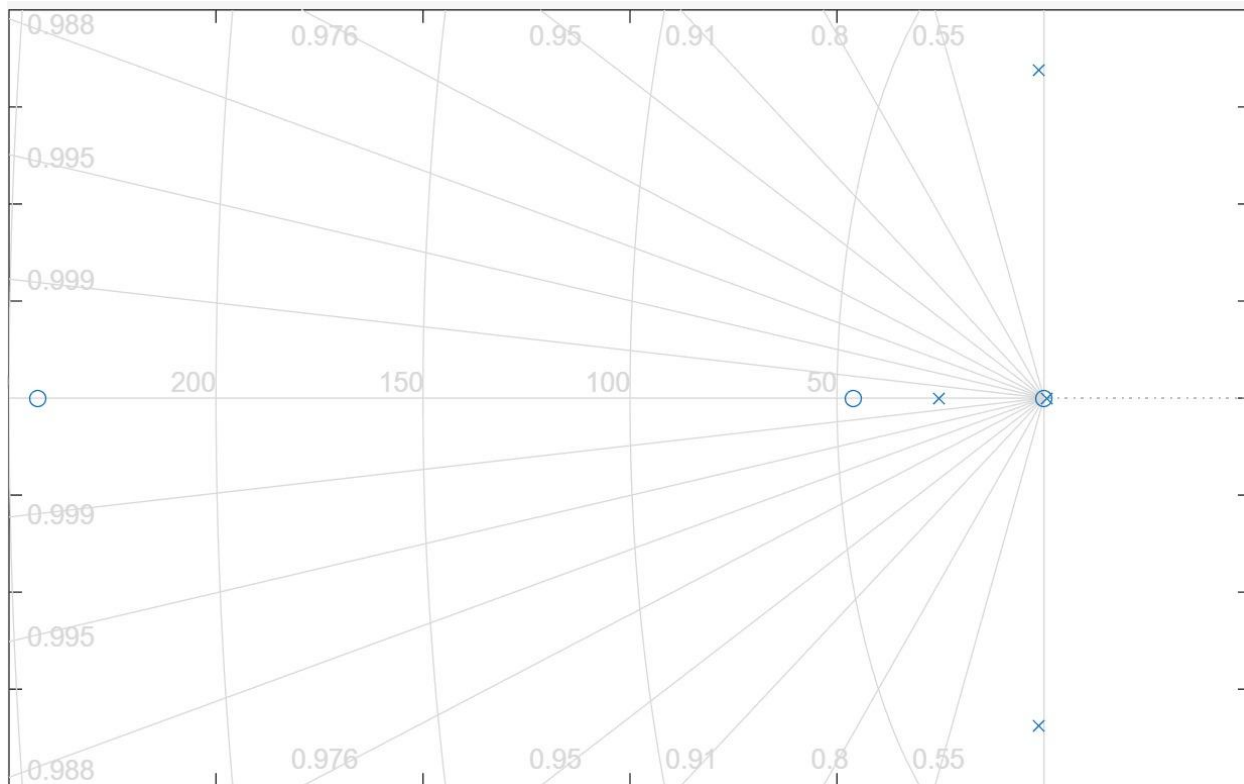
```
>> Warning: Simulation did not reach steady state. Please specify YFINAL if this system is stable and eventually settles.
> In DynamicSystem/stepinfo (line 74)
```

```
info =
```

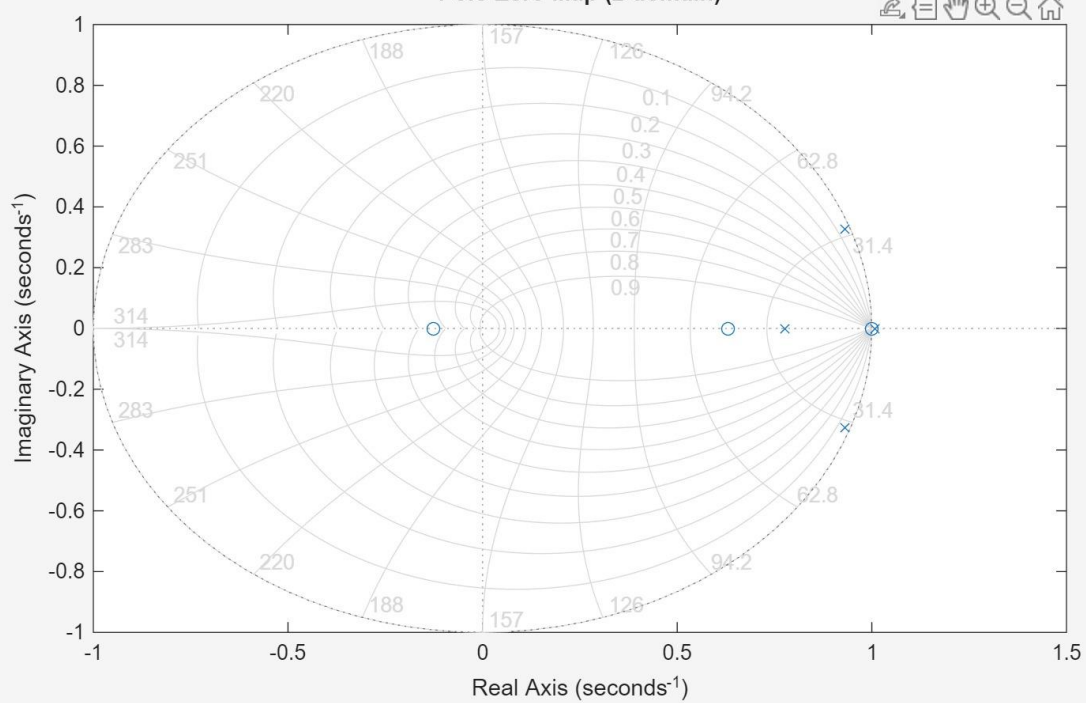
```
struct with fields:
```

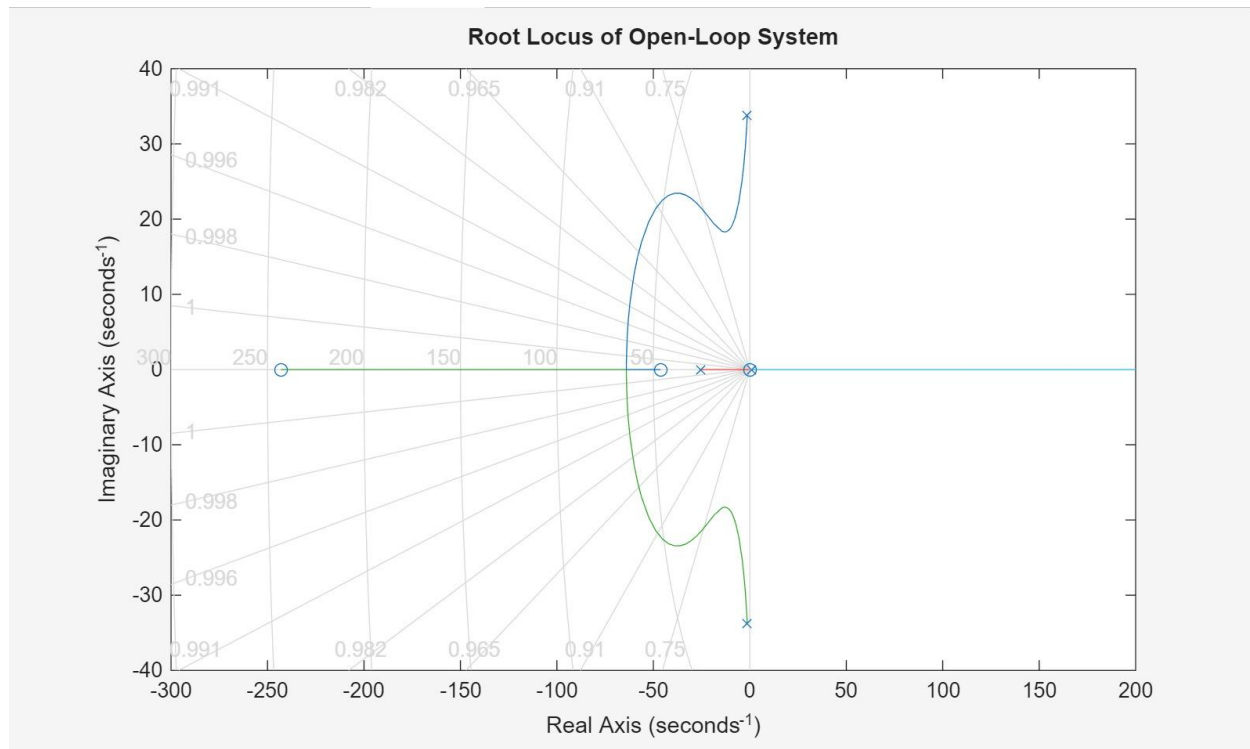
```
    RiseTime: NaN
  TransientTime: NaN
   SettlingTime: NaN
   SettlingMin: NaN
   SettlingMax: NaN
    Overshoot: NaN
   Undershoot: NaN
        Peak: Inf
    PeakTime: Inf|
```

Errors we got

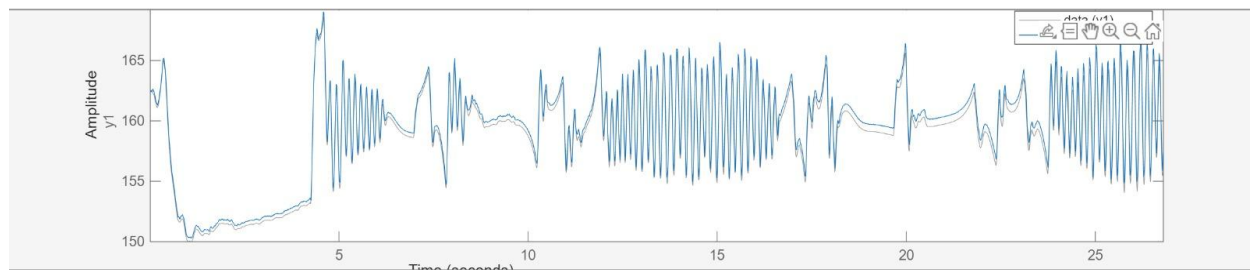


Pole-Zero Map (z-domain)

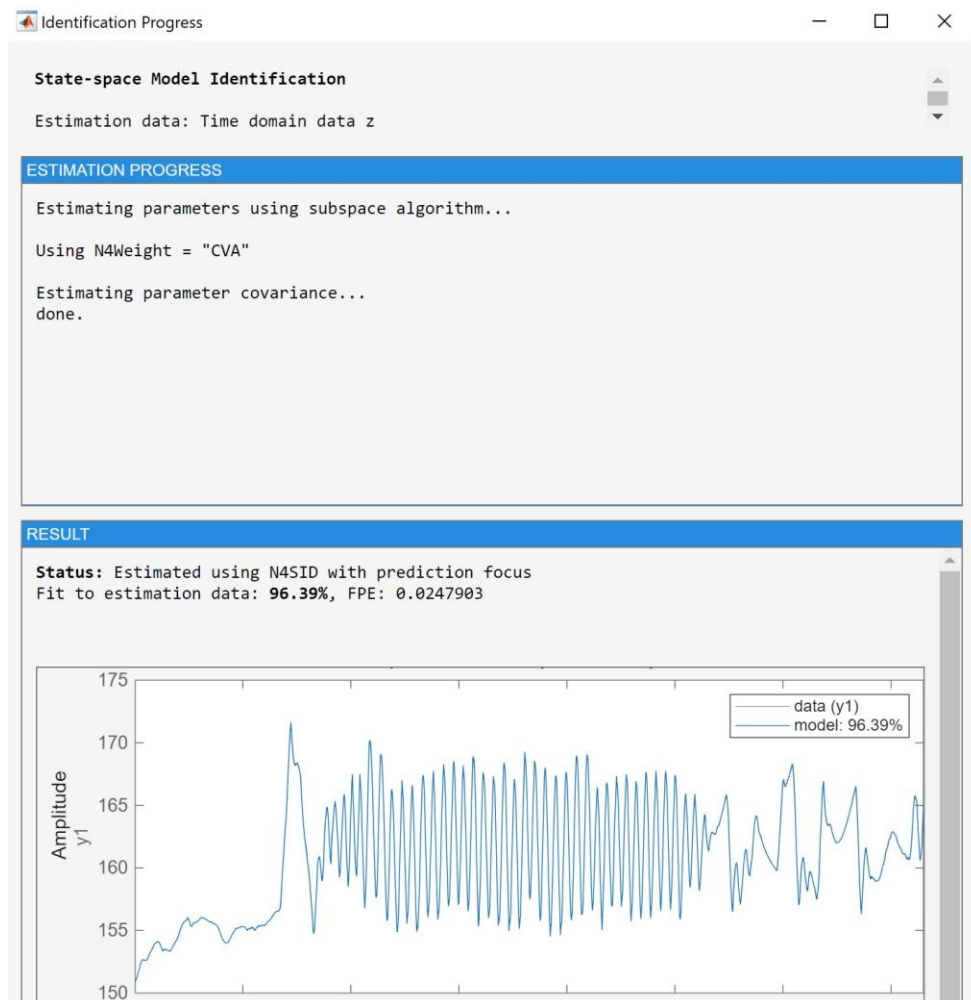




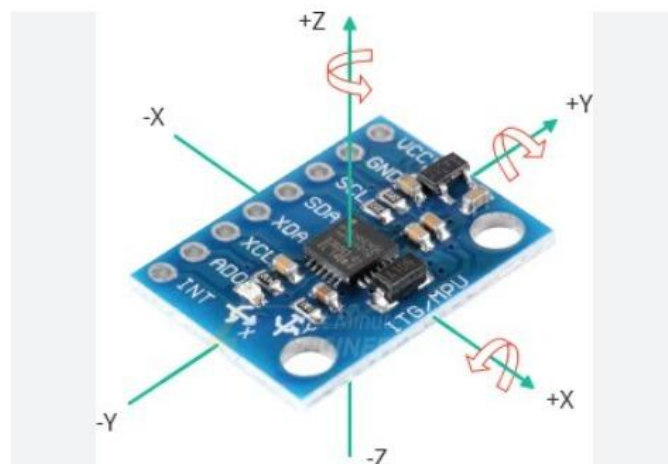
System Identification



Response when motor moved from its place indicating the sensitivity of any small adjustment



Best fit result we got



Rotation around y-axis

Videos

https://drive.google.com/drive/folders/1oe9k0c6Eu1Bi_46qxU5KsxWDuFWQp8gW?usp=drive_link